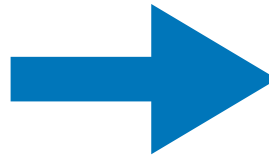
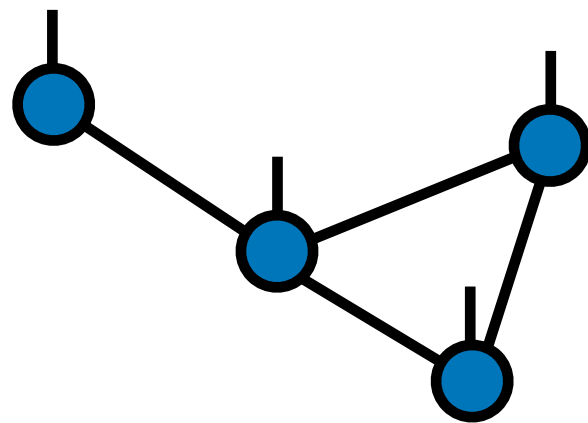


2: The ITensor Software

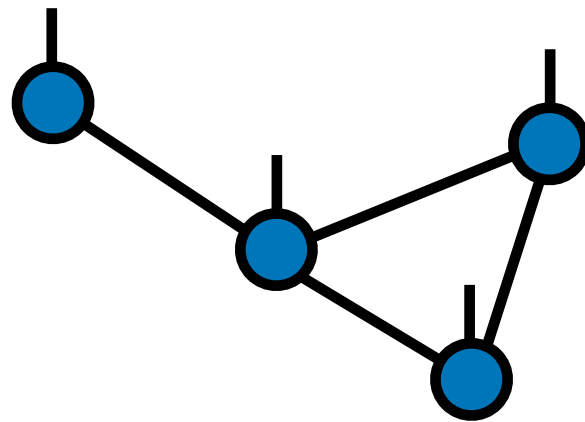


```
function hubbard_chain(sites; N, U=0.0, t2=0.0, V=0.0, kws...)
  @show U
  Hterms = itm.OpSum()
  for j in 1:(N - 1)
    Hterms += -1, "Cdagup", j, "Cup", j+1
    Hterms += -1, "Cdagup", j+1, "Cup", j
    Hterms += -1, "Cdagdn", j, "Cdn", j+1
    Hterms += -1, "Cdagdn", j+1, "Cdn", j
  end
end
```

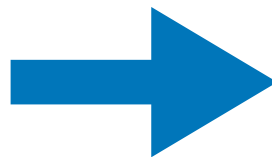
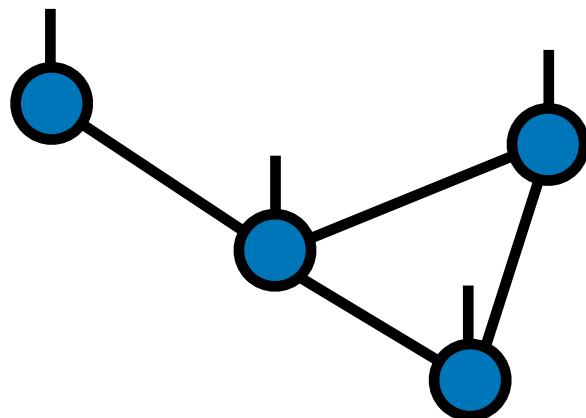
July 2025

Motivation

Tensor diagrams are a powerful notation for tensor networks and tensor contractions

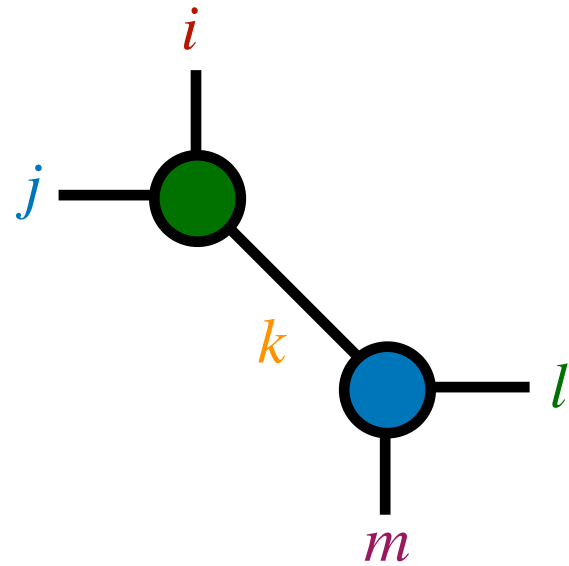


What if there was software modeled on tensor diagrams?



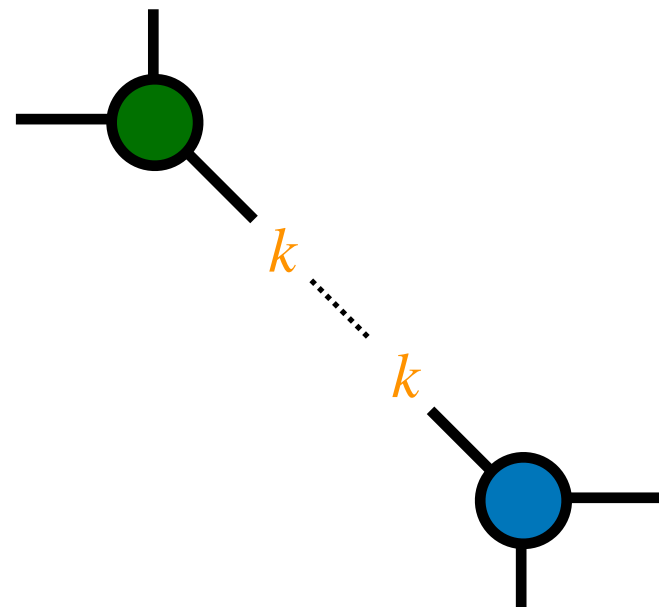
```
function hubbard_chain(sites; N, U=0.0, t2=0.0, V=0.0, kws...)
  @show U
  Hterms = itm.OpSum()
  for j in 1:(N - 1)
    Hterms += -1, "Cdagup", j, "Cup", j+1
    Hterms += -1, "Cdagup", j+1, "Cup", j
    Hterms += -1, "Cdagdn", j, "Cdn", j+1
    Hterms += -1, "Cdagdn", j+1, "Cdn", j
  end
end
```

Reminder: connecting index line means contraction

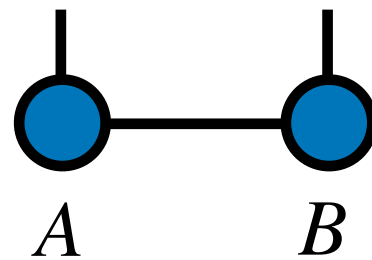


Which indices can connect?

Should be the **same** index – same size and meaning



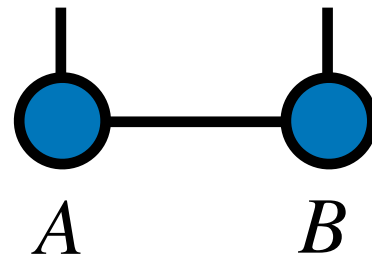
Some tensor libraries use index positions...



```
contract(A,{1,2},B,{2,3})
```

Must know contracted index is:
second index of A , first index of B

String identifiers help...



```
contract(A,{'i','j'},B,{'j','k'})
```

But the strings are temporary

Not unique – might repeat later

The ITensor Software

*Can indices "remember"
their identity?*

Before making an ITensor, we make indices

```
julia> i = Index(3)  
          (dim=3|id=807)
```

i
|

Before making an ITensor, we make indices

```
julia> i = Index(3)  
      (dim=3|id=807)
```

i
|



dimension unique identifier

Identifier lets index "recognize" itself

"Intelligent" index

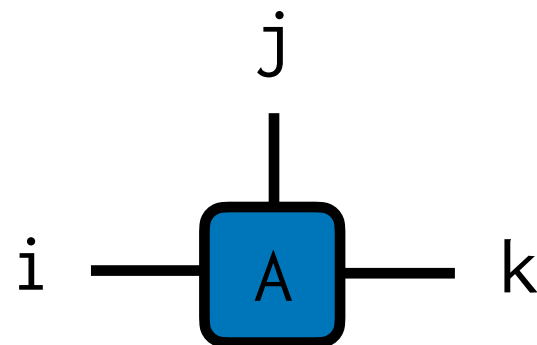
Making some indices

```
julia> i = Index(3)  
julia> j = Index(2)  
julia> k = Index(4)
```

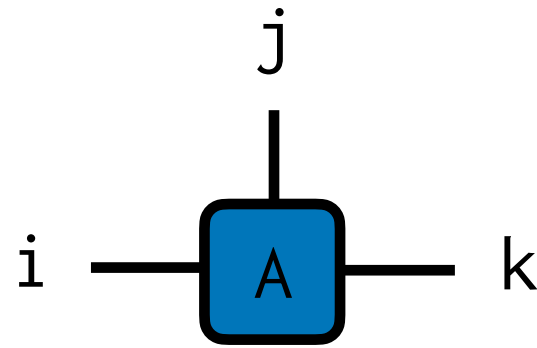
i j k
| | |

Now we can make tensors

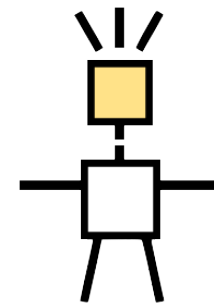
```
julia> A = ITensor(i,j,k)
```



```
julia> A = ITensor(i,j,k)
```



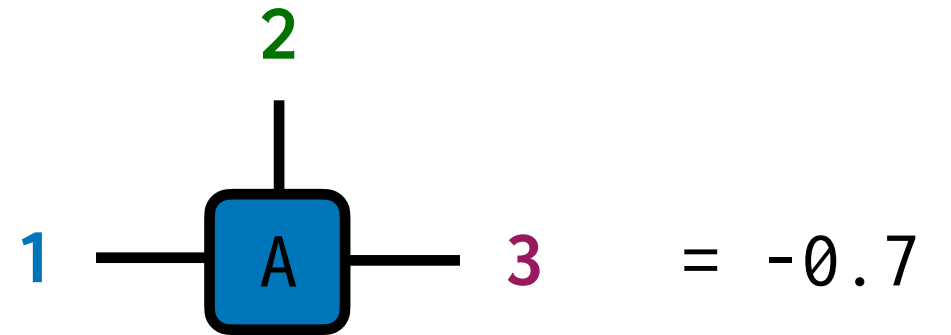
Indices are intelligent –
an intelligent tensor or **ITensor** *



* Not iTensor or i-Tensor ... 😊

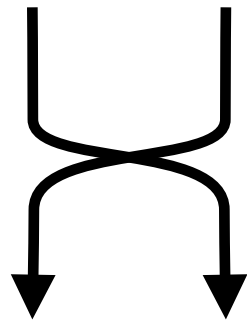
Setting element of an ITensor

```
julia> A[i=>1,j=>2,k=>3] = -0.7
```

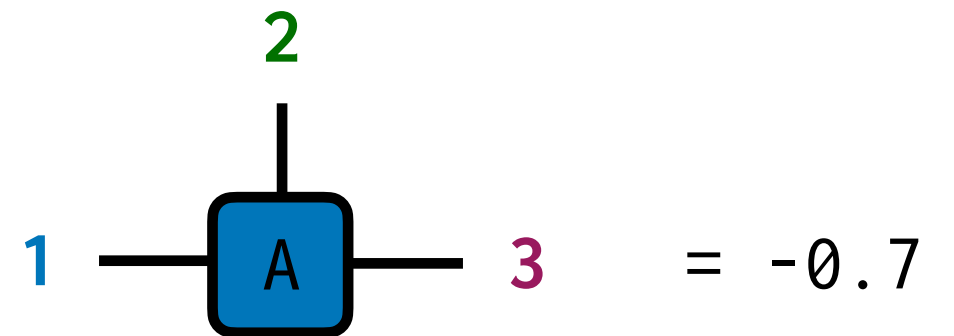


Any order allowed since indices known

```
julia> A[i=>1,j=>2,k=>3] = -0.7
```



```
julia> A[j=>2,i=>1,k=>3] = -0.7
```



Both commands have same effect

Operations with ITensors

Adding ITensors

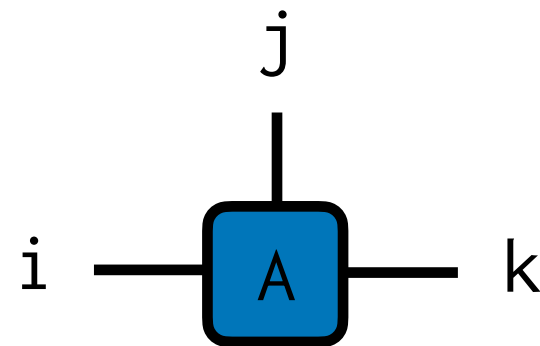
Assume we've made indices

```
julia> i = Index(3)
julia> j = Index(2)
julia> k = Index(4)
```

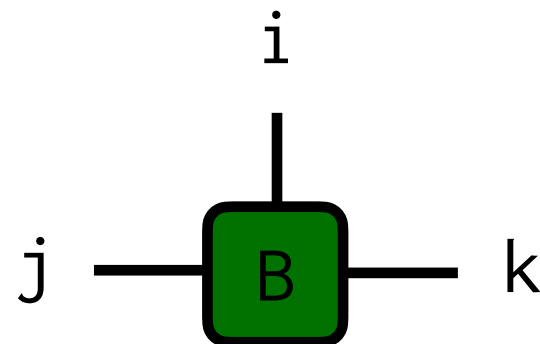
i j k
| | |

And ITensors, filled with elements (not shown)

```
julia> A = ITensor(i,j,k)
```

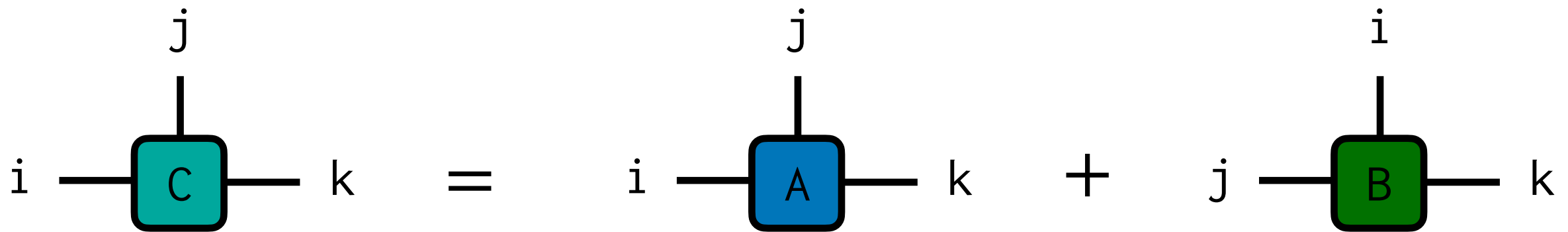


```
julia> B = ITensor(j,i,k)
```



Add ITensors with "+"

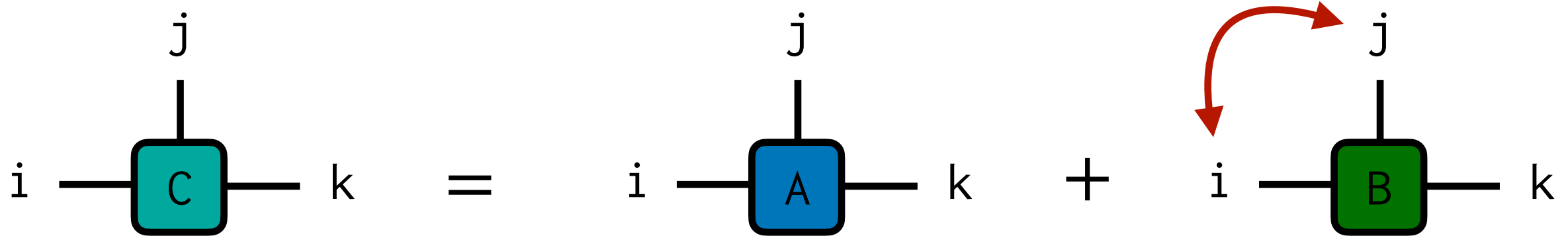
```
julia> C = A + B
```



Guaranteed correct, regardless of index ordering

Add ITensors with "+"

```
julia> C = A + B
```

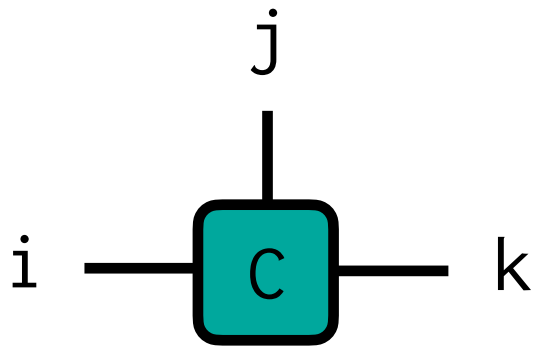


Guaranteed correct, regardless of index ordering

B copied and permuted before adding to A

Add ITensors with "+"

```
julia> C = A + B
```



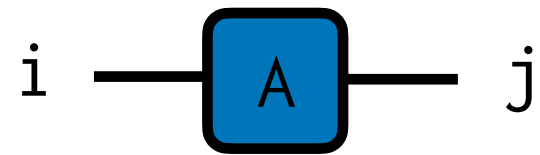
Guaranteed correct, regardless of index ordering

B copied and permuted before adding to A

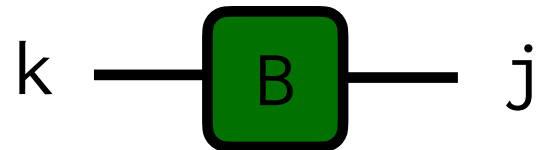
Contracting ITensors

Given two ITensors

```
julia> A = ITensor(i,j)
```



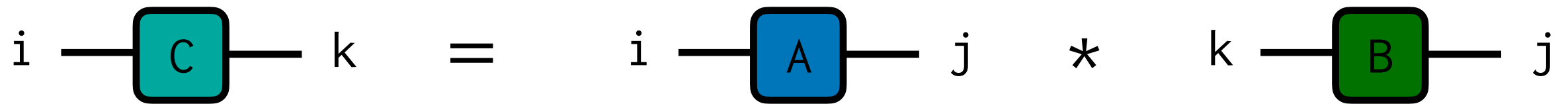
```
julia> B = ITensor(k,j)
```



How to contract matching indices?

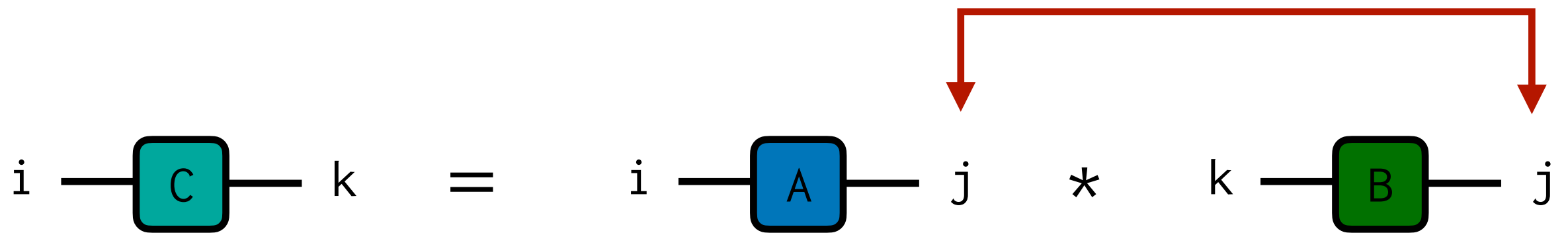
Contract ITensors with "*"

```
julia> C = A * B
```



Contract ITensors with "*"

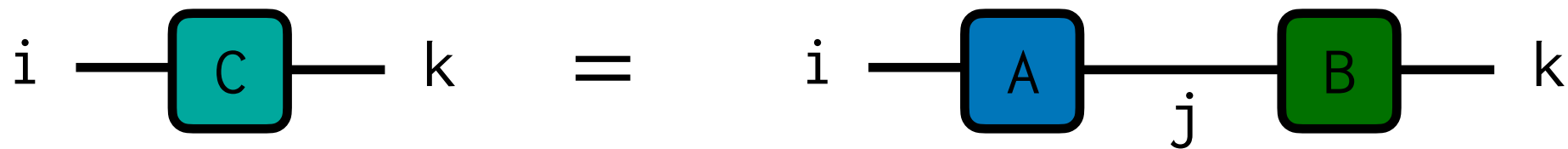
```
julia> C = A * B
```



Matching indices recognized

Contract ITensors with "*"

```
julia> C = A * B
```

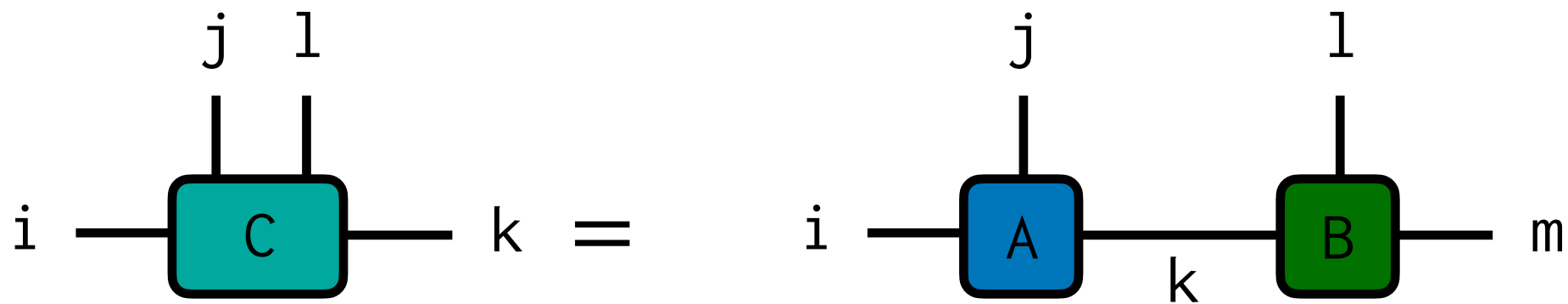


Matching indices recognized

Tensors permuted and contracted

General tensor contractions always

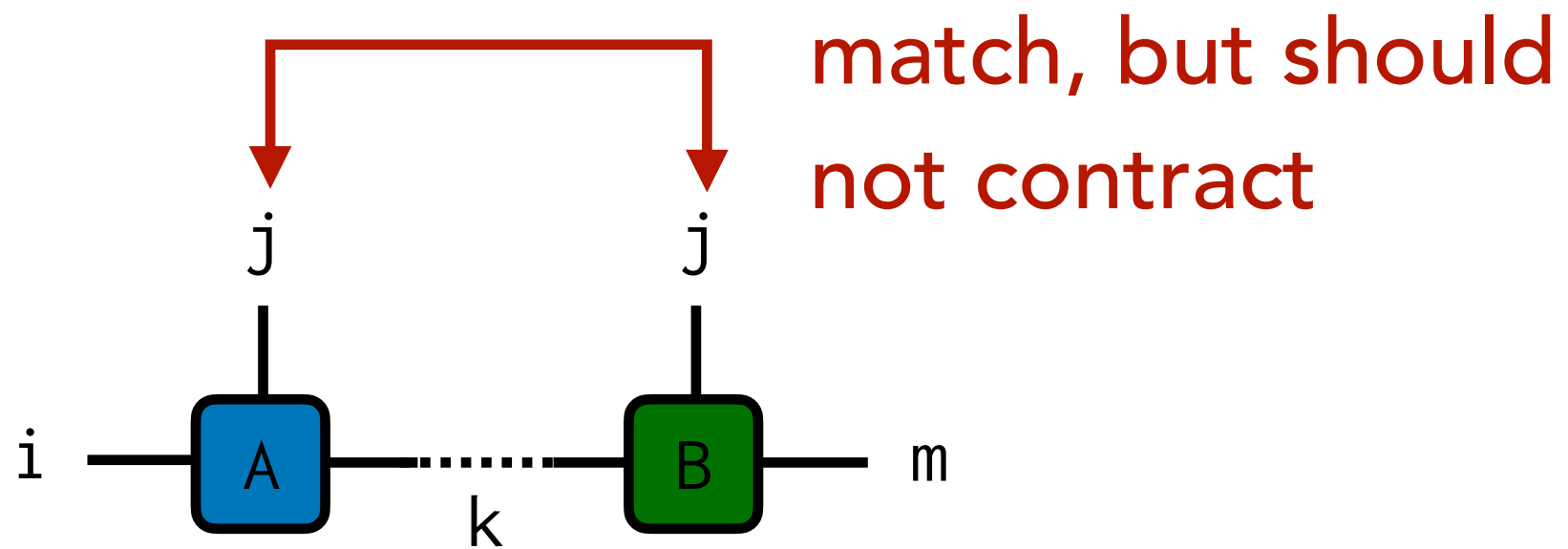
```
julia> C = A * B
```



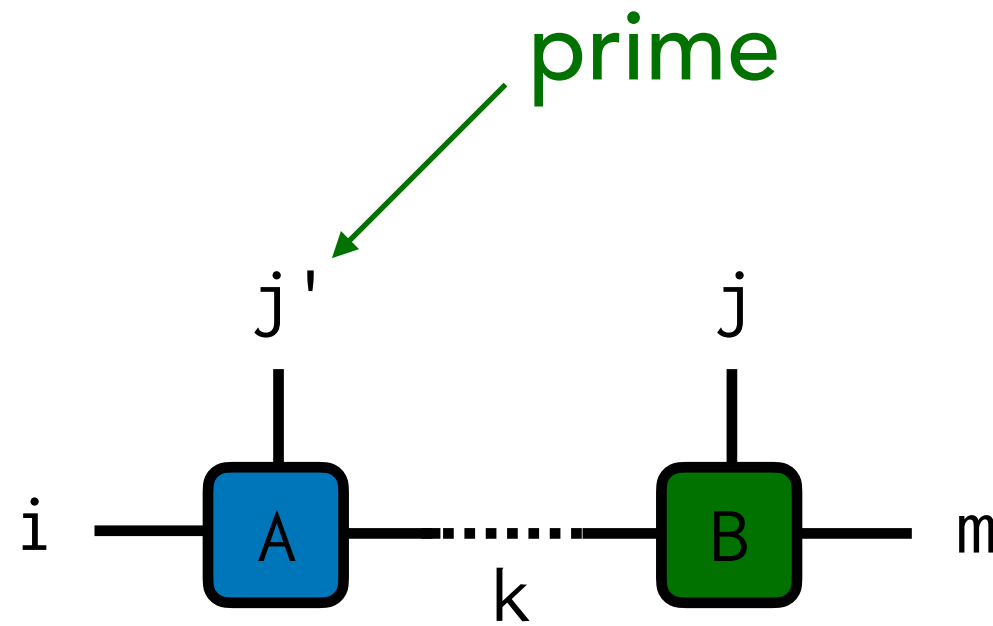
Assuming all matching indices should be contracted

Priming Indices

What if certain matching indices should stay uncontracted?

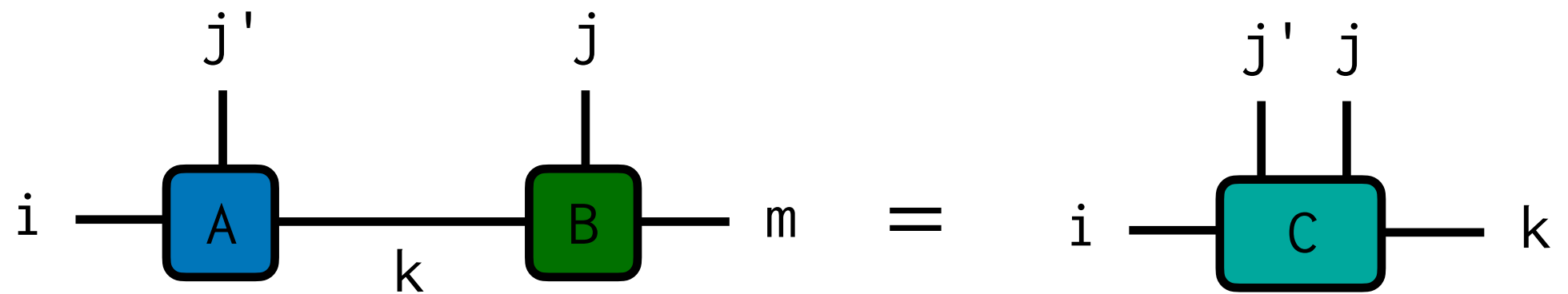


Prime an index to make it different



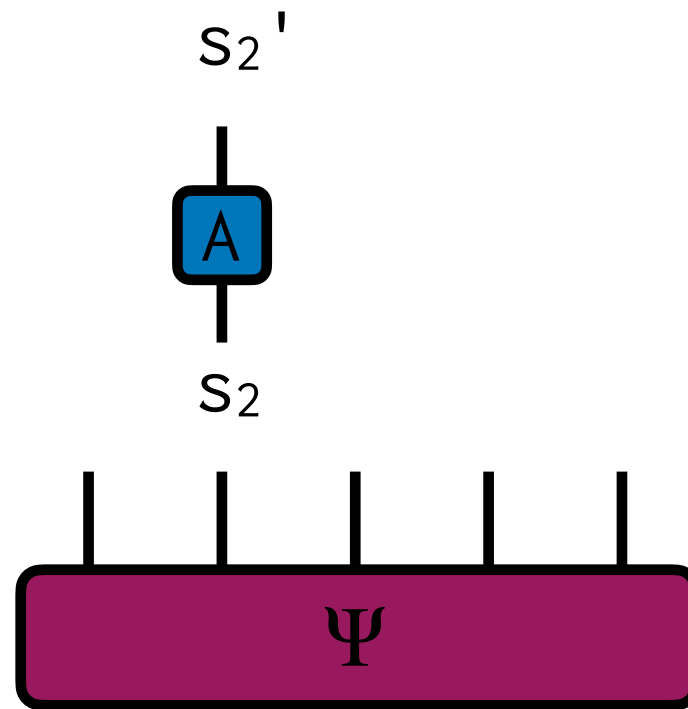
```
julia> prime(A,j) # prime index j only
```

Contracting now gives desired result



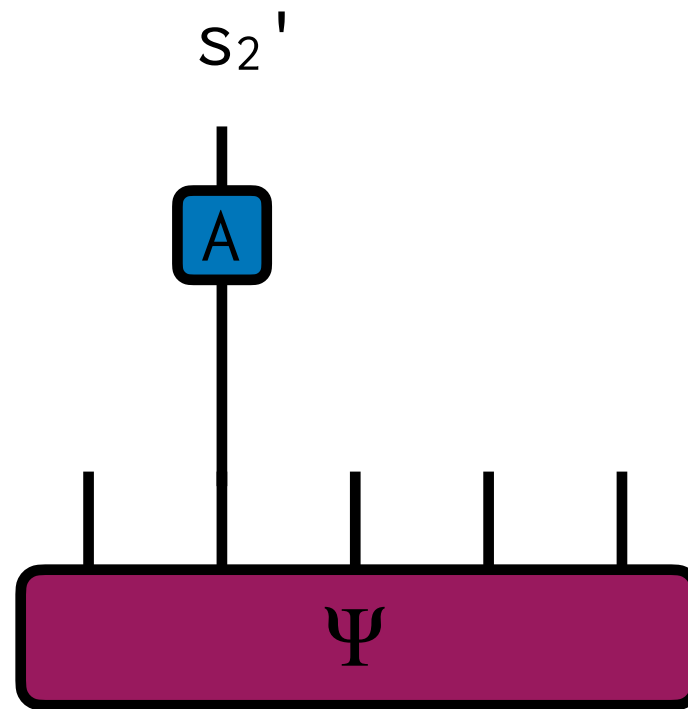
```
julia> Aprime = prime(A,j) # prime index j only  
julia> C = Aprime * B
```

Key use of priming is applying operators



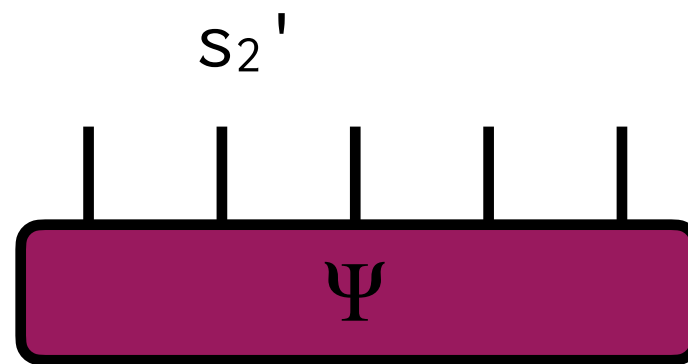
```
julia> A_psi = A * psi
```

Key use of priming is applying operators



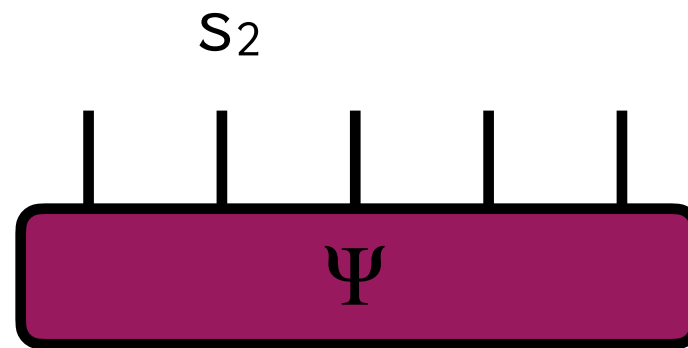
```
julia> A_psi = A * psi
```

Remove prime with `noprime` function



```
julia> A_psi = A * psi  
julia> A_psi = noprime(A_psi)
```

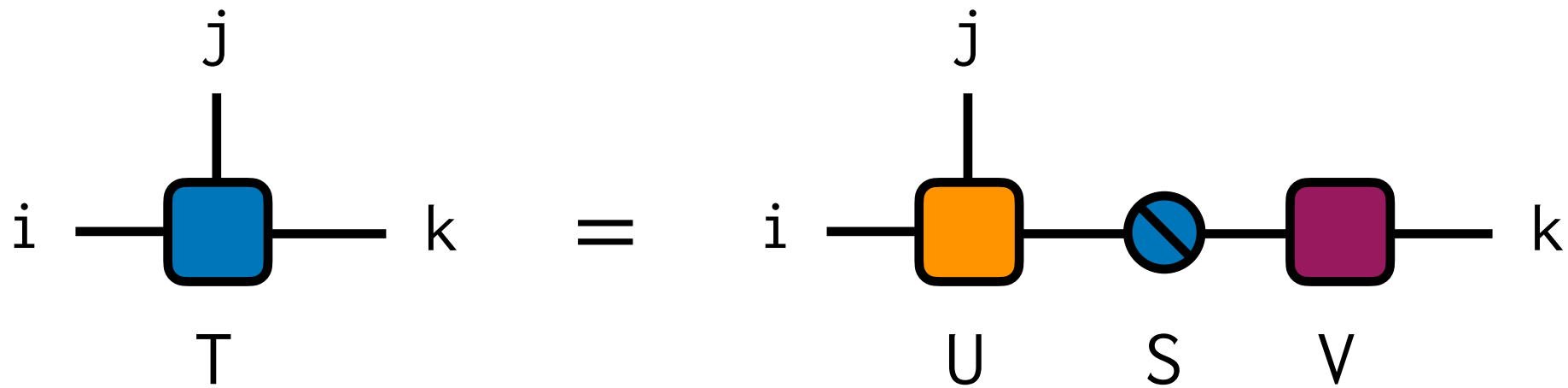
Remove prime with `noprime` function



```
julia> A_psi = A * psi  
julia> A_psi = noprime(A_psi)
```

ITensor Decompositions

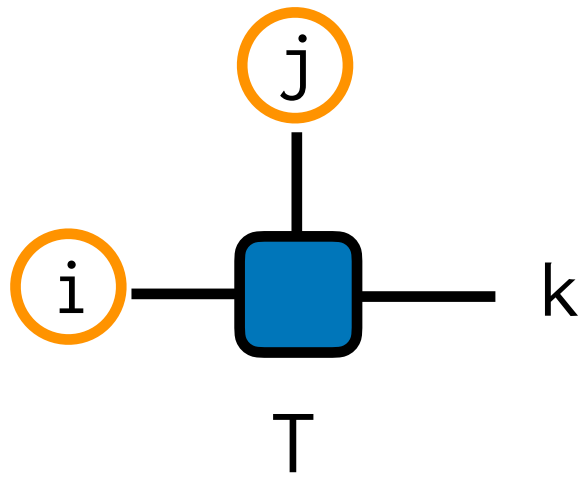
Recall we can SVD a tensor



```
julia> T ≈ U * S * V  
true
```

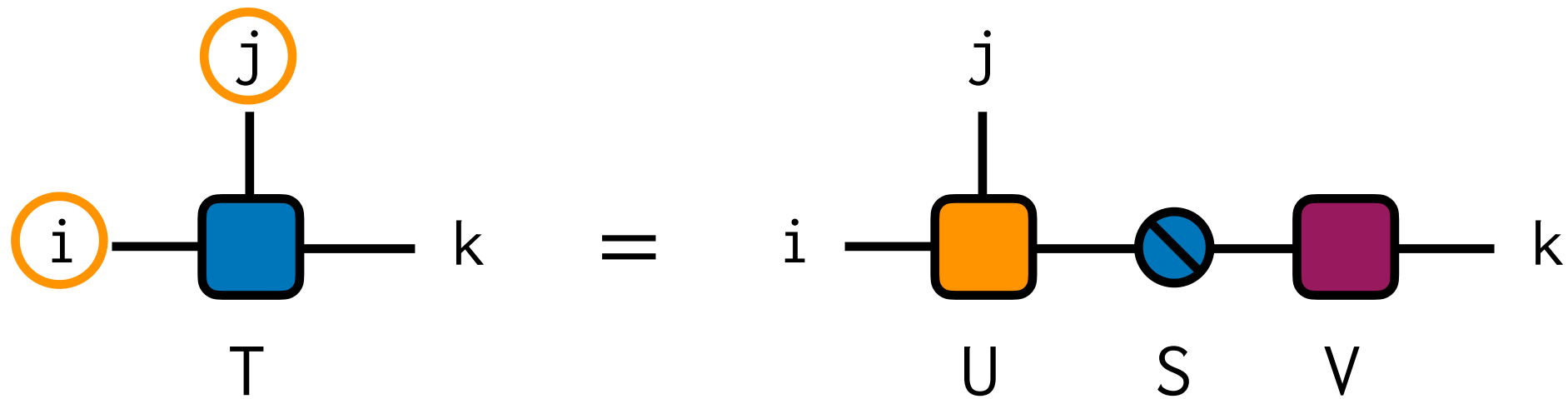
How to SVD an ITensor?

Select which indices go onto "U"



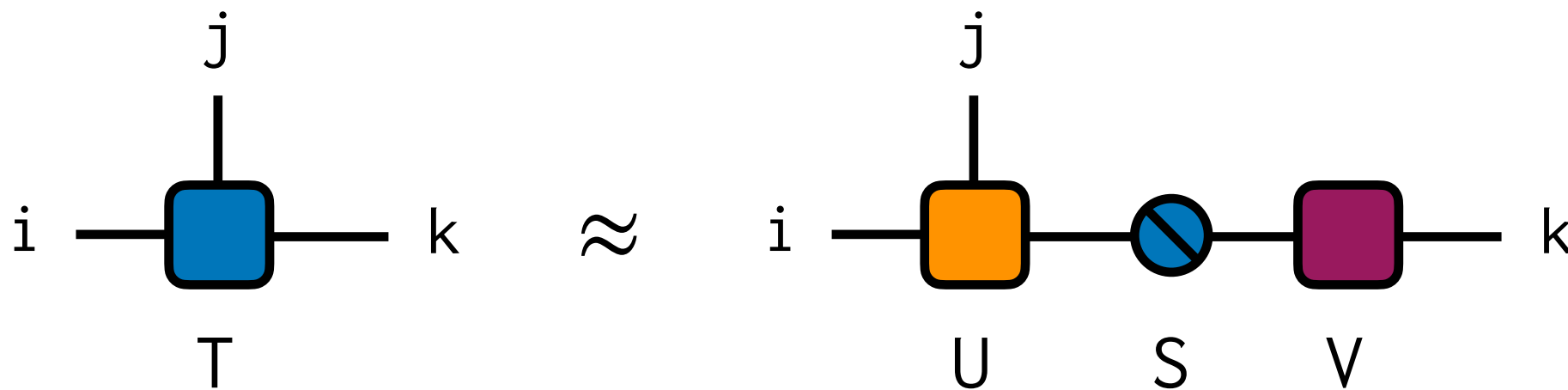
```
julia> u_inds = [i,j]
```

Call ITensor `svd` function



```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds)
```

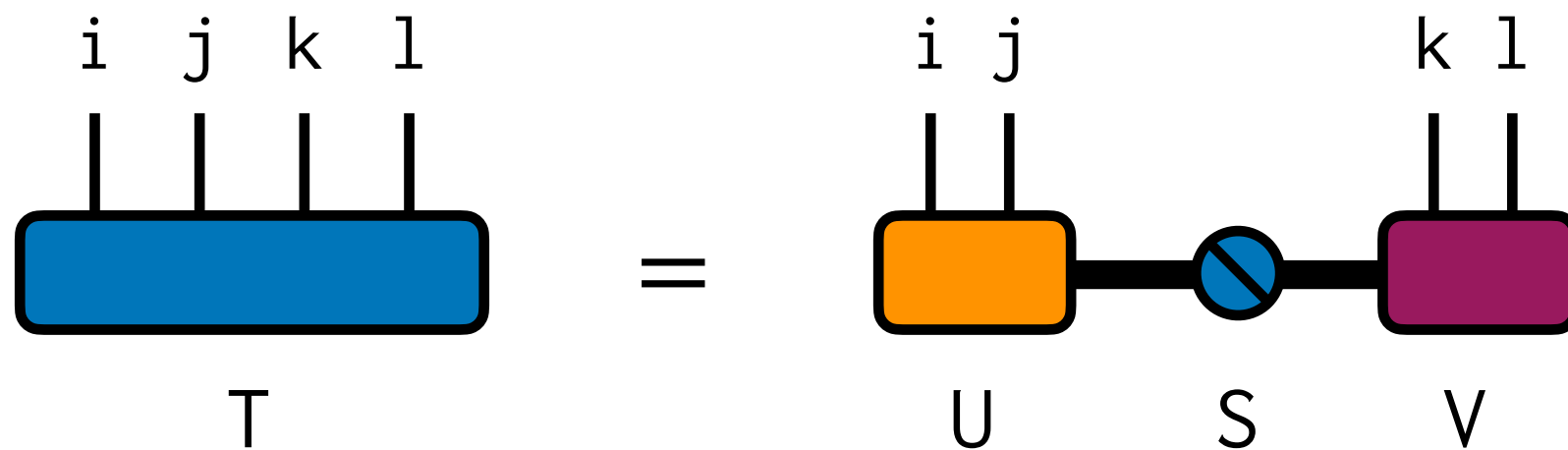
To compute truncated SVD, pass
truncation parameters: maxdim, cutoff



```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds; maxdim=10, cutoff=1E-4)
```

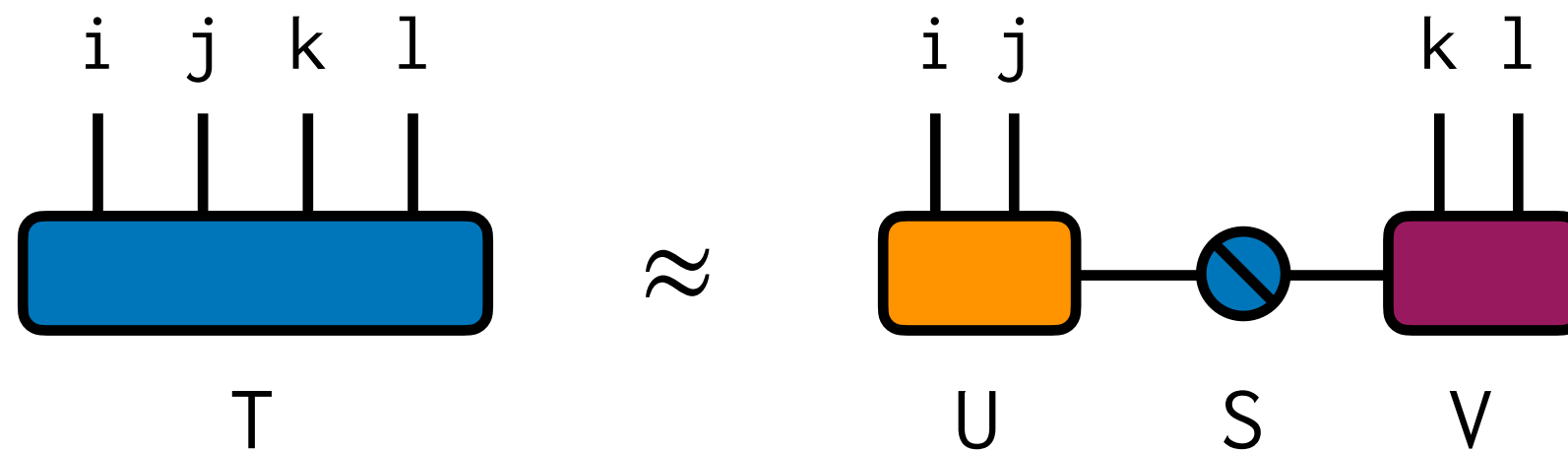
- maxdim: maximum dimension of internal bond
- cutoff ϵ : discard singular values λ_j such that $\sum_{j \in \text{discarded}} \lambda_j^2 \leq \epsilon$

Intuition about truncation - large outer spaces
yet tensor has hidden "low rank" property



```
julia> u_inds = [i,j]  
julia> U,S,V = svd(T, u_inds)
```

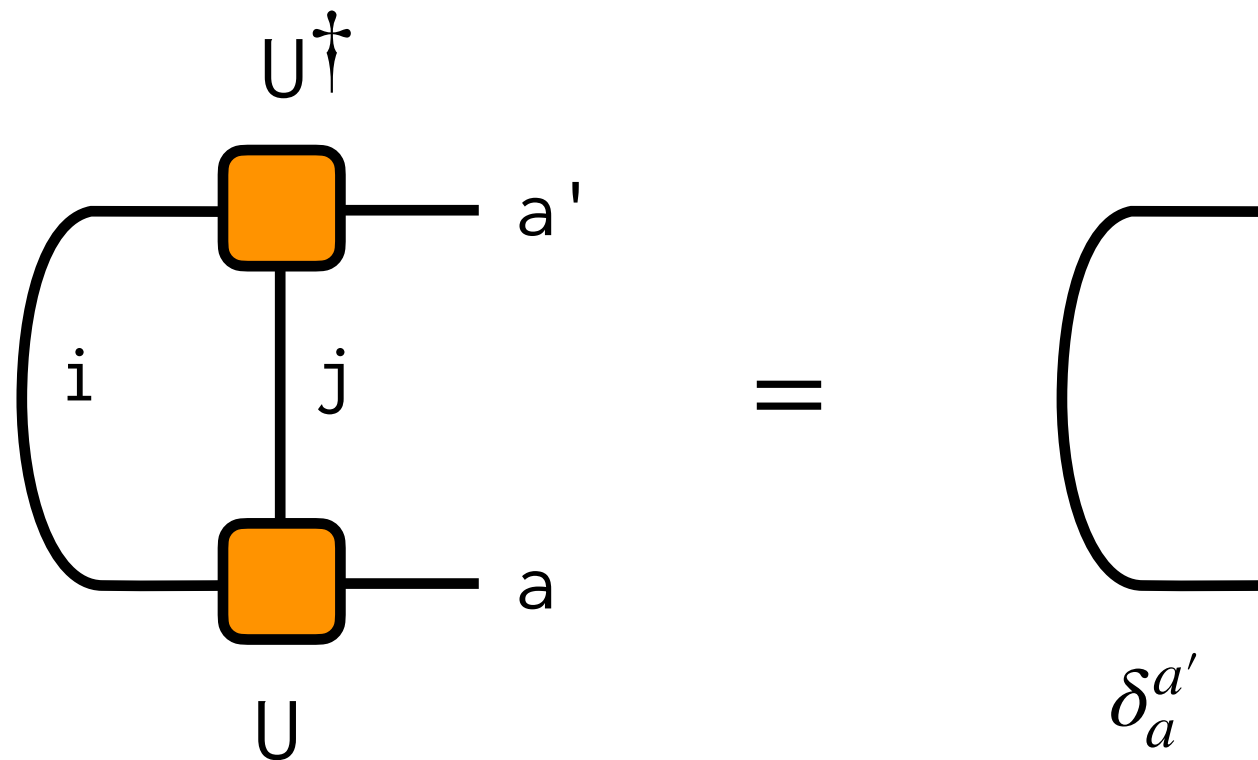
Intuition about truncation - large outer spaces
yet tensor has hidden "low rank" property



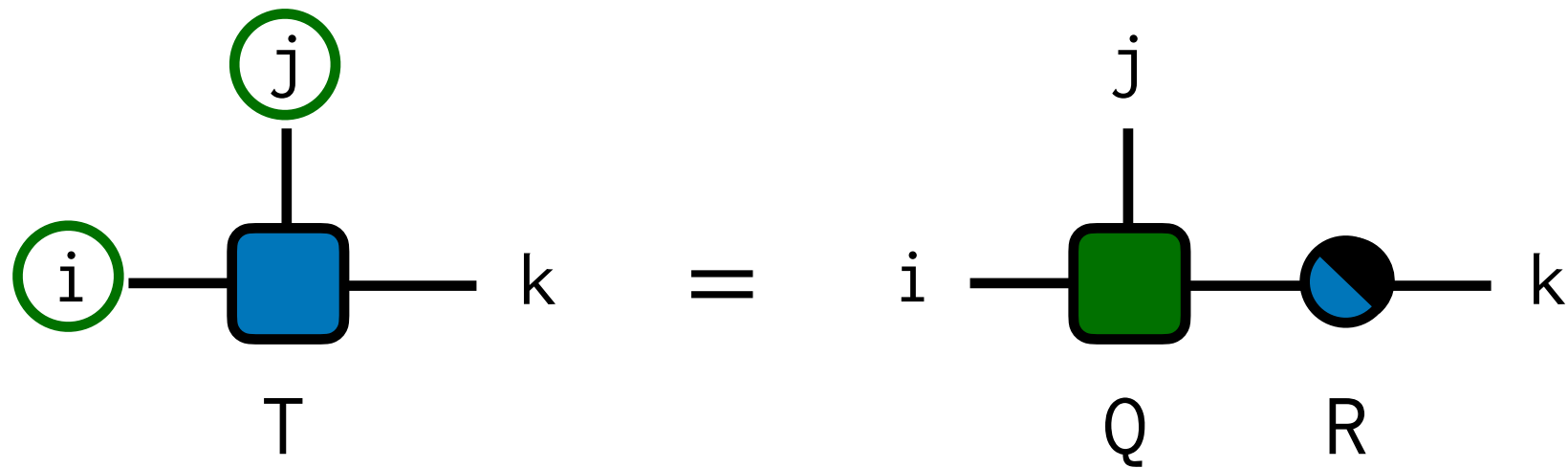
```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds; maxdim=10, cutoff=1E-4)
```

U and V have "isometric" property

One-sided 'unitary':

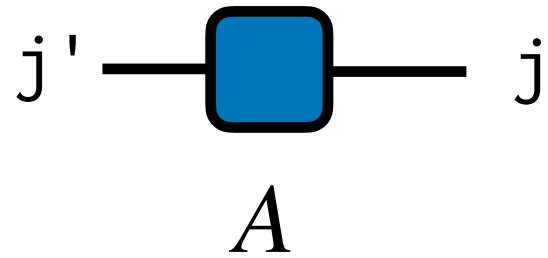


Other decompositions work similarly
QR, for example



```
julia> q_inds = [i,j]
julia> Q,R = qr(T, q_inds)
julia> T ≈ Q * R
true
```

Also exponentiation of ITensors



A diagram representing an ITensor e^{tA} . It consists of a green square with rounded corners and a black border. A horizontal line extends from the left side of the square to the label j' , and another horizontal line extends from the right side to the label j . The label e^{tA} is centered below the square.

$$e^{tA} \approx I + tA + \frac{t^2}{2!}A^2 + \dots$$

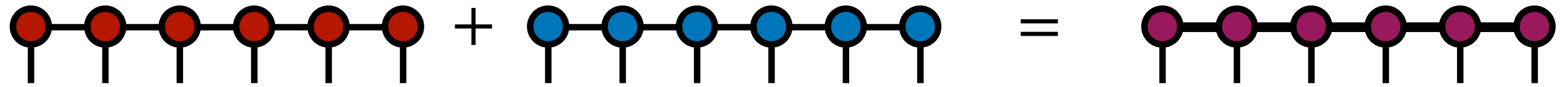
```
julia> A = ITensor(j',j)
```

```
...
```

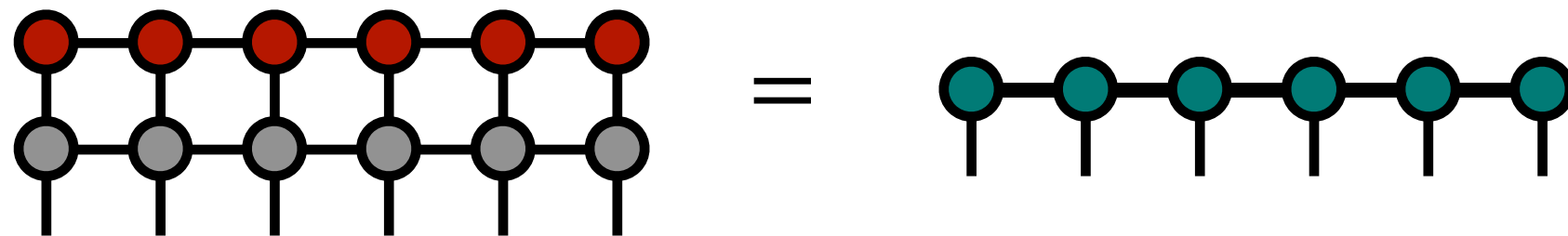
```
julia> expH = exp(t * A)
```

Matrix Product State (MPS) Algorithms

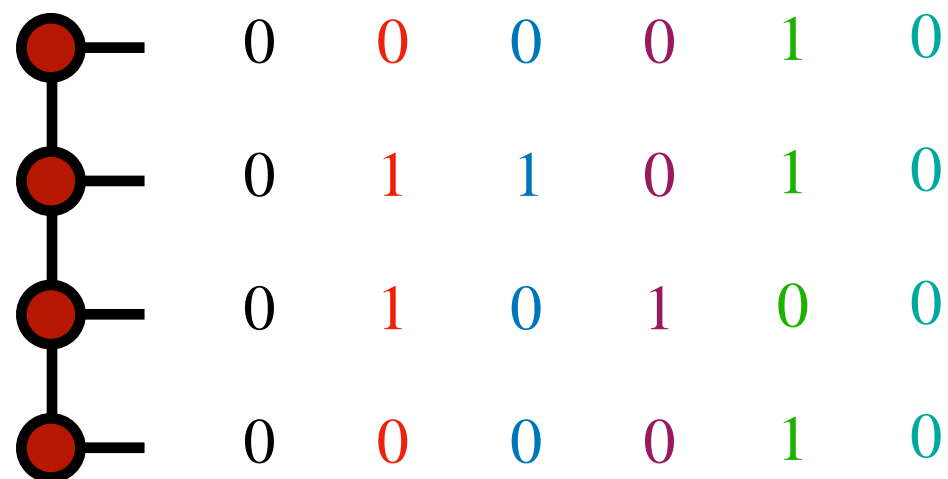
Can efficiently sum MPS in compressed form:

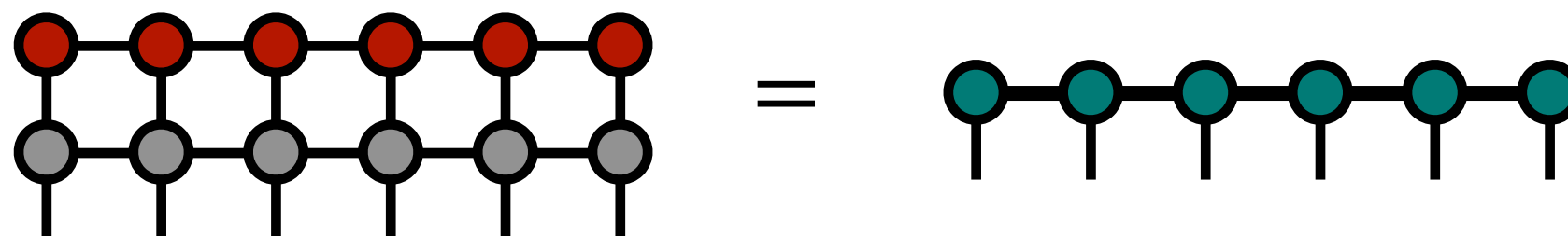


multiply by other networks:



and perfectly sample:





matrix product operator (MPO) network

MPS Algorithms

We offer MPS and "MPO" algorithms through the
ITensorMPS
package

```
using ITensors  
using ITensorMPS
```

MPS Algorithms

The ITensorMPS package offers many helpful algorithms for working with MPS and MPO's

- ❑ **OpSum** system – making MPOs from operators
- ❑ **dmrg** & **tdvp** – computing ground states and dynamics
- ❑ **expect** – computing expected values of operators
- ❑ **correlation_matrix** – compute correlation functions
- ❑ **inner** – overlap MPS and MPOs
- ❑ **contract**, **sum** – algebra of MPS and MPO

MPS Algorithms

OpSum – powerful "domain-specific language" (DSL)
for making MPOs from math expressions

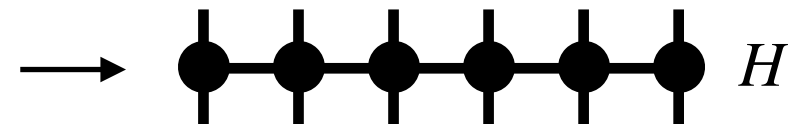
$$\sum_j S_j^z S_{j+1}^z$$



```
sites = siteinds("S=1/2",N)

terms = OpSum()
for j=1:N-1
  terms += "Sz",j, "Sz",j+1
end

H = MPO(terms,sites)
```



MPS Algorithms

Starting from beginning: first make an array of "sites"
Just a Julia array of Index objects

```
sites = siteinds("S=1/2",N)
```

| | | | | |

MPS Algorithms

Next fill up OpSum with "terms" of the operator

```
sites = siteinds("S=1/2",N)
```

```
terms = OpSum()
```

```
for j=1:N-1
```

```
    terms += "Sz",j, "Sz",j+1
```

```
end
```

| | | | | |

Internal data of "terms" similar to:

(1.0,"Sz",1,"Sz",2), (1.0,"Sz",2,"Sz",3), ...

MPS Algorithms

Finally, construct MPO – compresses terms together [1,2]

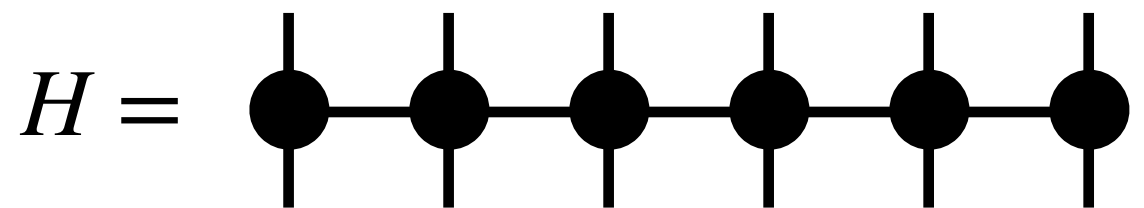
```
sites = siteinds("S=1/2",N)

terms = OpSum()
for j=1:N-1
    terms += "Sz",j, "Sz",j+1
end

H = MPO(terms,sites)
```

| | | | | |

(1.0,"Sz",1,"Sz",2), (1.0,"Sz",2,"Sz",3), ...



Optimal "bond dimension" typically reached

[1] Chan, Keselman, Nakatani, Li, White, J. Chem. Phys 145 (2016)

[2] Parker, Zaletel, Phys. Rev. B, 102, 035147 (2020)

MPS Algorithms

Very wide range of operators can be made

$$H = \sum_j S_j^z S_{j+1}^z$$



```
sites = siteinds("S=1/2",N)
```

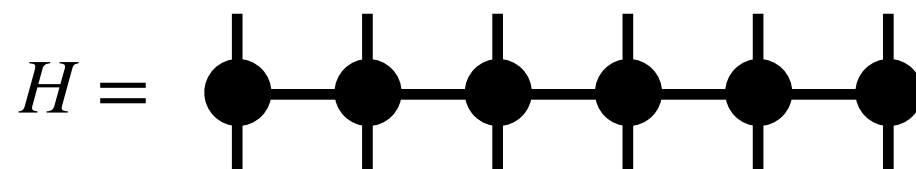
```
terms = OpSum()
```

```
for j=1:N-1
```

```
    terms += "Sz",j, "Sz",j+1
```

```
end
```

```
H = MPO(terms,sites)
```



MPS Algorithms

Very wide range of operators can be made

$$H = \sum_j S_j^z S_{j+1}^z + \frac{1}{2} S_j^+ S_{j+1}^- + \frac{1}{2} S_j^- S_{j+1}^+$$



```
sites = siteinds("S=1/2",N)
```

```
terms = OpSum()
```

```
for j=1:N-1
```

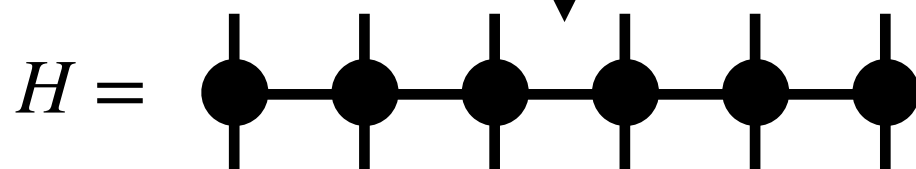
```
    terms += "Sz",j, "Sz",j+1
```

```
    terms += 1/2,"S+",j, "S-",j+1
```

```
    terms += 1/2,"S-",j, "S+",j+1
```

```
end
```

```
H = MPO(terms,sites)
```



MPS Algorithms

Changing site type automatically gives correct operators

$$H = \sum_j S_j^z S_{j+1}^z + \frac{1}{2} S_j^+ S_{j+1}^- + \frac{1}{2} S_j^- S_{j+1}^+$$



```
sites = siteinds("S=1/2",N)
```

```
terms = OpSum()
```

```
for j=1:N-1
```

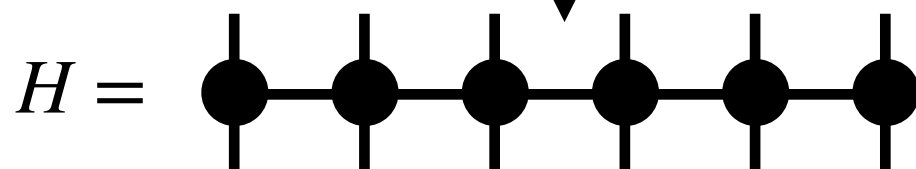
```
    terms += "Sz",j, "Sz",j+1
```

```
    terms += 1/2,"S+",j, "S-",j+1
```

```
    terms += 1/2,"S-",j, "S+",j+1
```

```
end
```

```
H = MPO(terms,sites)
```



MPS Algorithms

Changing site type automatically gives correct operators

$$H = \sum_j S_j^z S_{j+1}^z + \frac{1}{2} S_j^+ S_{j+1}^- + \frac{1}{2} S_j^- S_{j+1}^+$$



```
sites = siteinds("S=1",N)
```

```
terms = OpSum()
```

```
for j=1:N-1
```

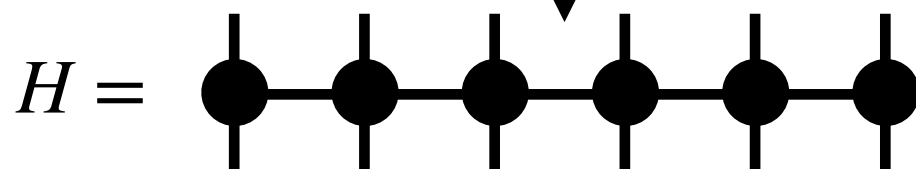
```
    terms += "Sz",j, "Sz",j+1
```

```
    terms += 1/2,"S+",j, "S-",j+1
```

```
    terms += 1/2,"S-",j, "S+",j+1
```

```
end
```

```
H = MPO(terms,sites)
```



MPS Algorithms

Particles (bosons, fermions) possible too

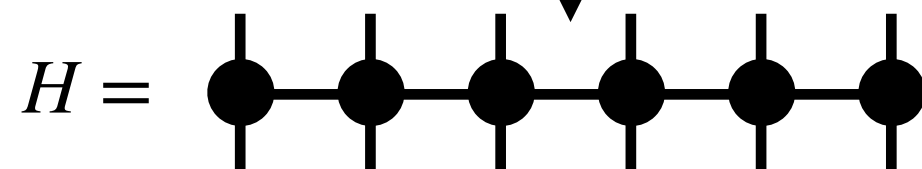
$$H = \sum_j c_j^\dagger c_{j+1} + c_{j+1}^\dagger c_j$$



```
sites = siteinds("Fermion",N)

terms = OpSum()
for j=1:N-1
    terms += "Cdag",j, "C",j+1
    terms += "C",j+1, "Cdag",j+1
end

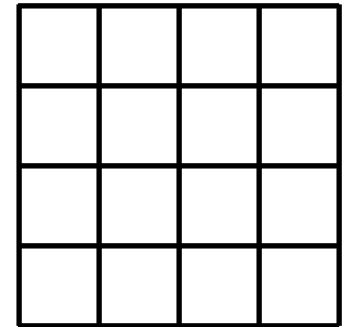
H = MPO(terms,sites)
```



MPS Algorithms

And quasi-two-dimensional systems

$$H = \sum_{\langle ij \rangle} S_i^z S_j^z + \frac{1}{2} S_i^+ S_j^- + \frac{1}{2} S_i^- S_j^+$$



lattice = square_lattice(Nx, Ny; yperiodic=false)

terms = OpSum()

for b in lattice

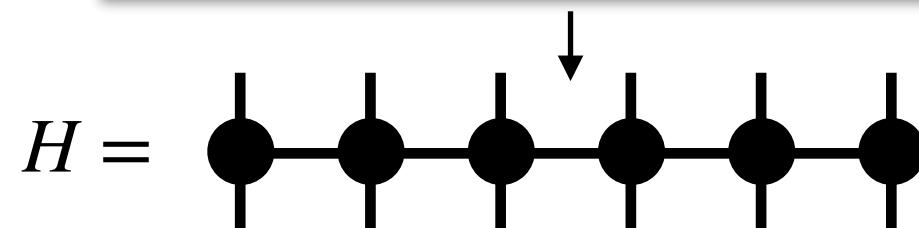
terms += "Sz", b.s1, "Sz", b.s2

terms += 1/2, "S+", b.s1, "S-", b.s2

terms += 1/2, "S-", b.s1, "S+", b.s2

end

H = MPO(terms, sites)



MPS Algorithms

Next let's look at **dmrg** and **tdvp**

- ☑ OpSum system – making MPOs from operators
- ☐ **dmrg & tdvp – computing ground states and dynamics**
- ☐ expect – computing expected values of operators
- ☐ correlation_matrix – compute correlation functions
- ☐ inner – overlap MPS and MPOs
- ☐ contract, sum – algebra of MPS and MPO

MPS Algorithms

ITensorMPS offers "black box" DMRG algorithm

```
using ITensors, ITensorMPS
```

```
N = 100
```

```
sites = siteinds("S=1", N)
```

```
terms = OpSum()
```

```
for j in 1:(N - 1)
```

```
    terms += "Sz", j, "Sz", j + 1
```

```
    terms += 0.5, "S+", j, "S-", j + 1
```

```
    terms += 0.5, "S-", j, "S+", j + 1
```

```
end
```

```
H = MPO(terms, sites)
```

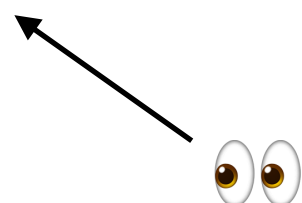
```
psi0 = random_mps(sites; linkdims=10)
```

```
nsweeps = 5
```

```
maxdim = [10, 20, 100, 100, 200]
```

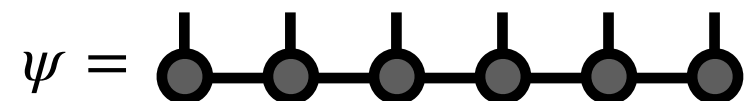
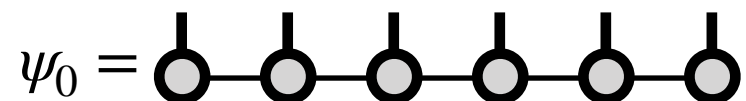
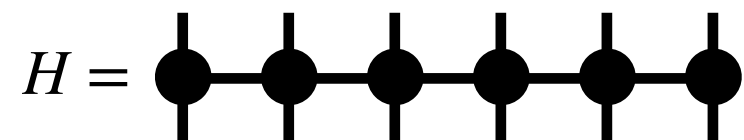
```
cutoff = [1E-11]
```

```
energy, psi = dmrg(H, psi0; nsweeps, maxdim, cutoff)
```



$$H = \sum_j S_j^z S_{j+1}^z + \frac{1}{2} S_j^+ S_{j+1}^- + \frac{1}{2} S_j^- S_{j+1}^+$$

(1.0, "Sz", 1, "Sz", 2), (1.0, "Sz", 2, "Sz", 3), ...



MPS Algorithms

TDVP is similar, for time evolution by some H

```
using ITensors, ITensorMPS
```

```
N = 100
```

```
sites = siteinds("S=1/2", N)
```

```
terms = OpSum()
```

```
for j in 1:(N - 1)
```

```
    terms += "Sz", j, "Sz", j + 1
```

```
    terms += 0.5, "S+", j, "S-", j + 1
```

```
    terms += 0.5, "S-", j, "S+", j + 1
```

```
end
```

```
H = MPO(terms, sites)
```

```
psi0 = random_mps(sites; linkdims=10)
```

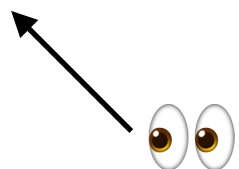
```
t = 10
```

```
time_step = 0.1
```

```
maxdim = 200
```

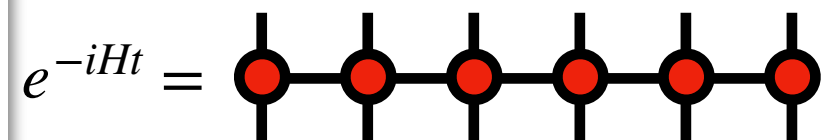
```
cutoff = 1E-10
```

```
psi_t = tdvp(H, -im*t, psi0; maxdim, cutoff, time_step)
```



$$H = \sum_j S_j^z S_{j+1}^z + \frac{1}{2} S_j^+ S_{j+1}^- + \frac{1}{2} S_j^- S_{j+1}^+$$

(1.0, "Sz", 1, "Sz", 2), (1.0, "Sz", 2, "Sz", 3), ...



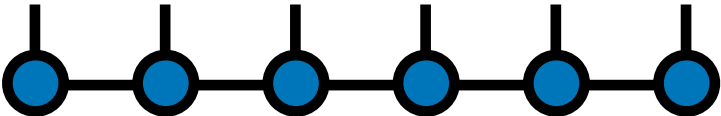
MPS Algorithms

Can use **expect**, **correlation_matrix**, and **inner** to analyze MPS

- ☑ OpSum system – making MPOs from operators
- ☑ dmrg & tdvp – computing ground states and dynamics
- ☐ **expect** – computing expected values of operators
- ☐ **correlation_matrix** – compute correlation functions
- ☐ **inner** – overlap MPS and MPOs
- ☐ contract, sum – algebra of MPS and MPO

MPS Algorithms

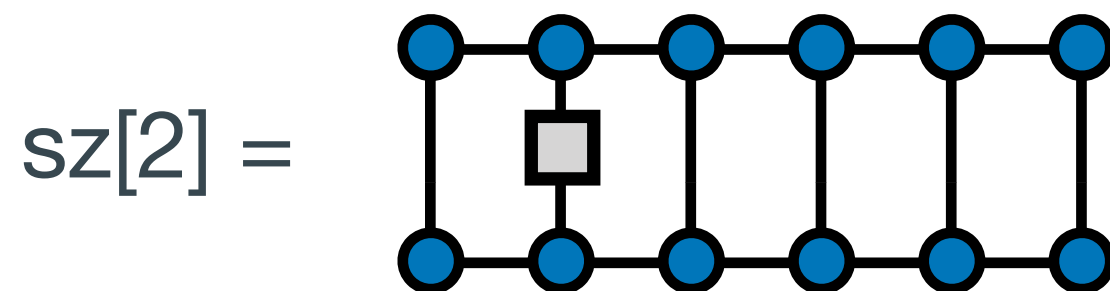
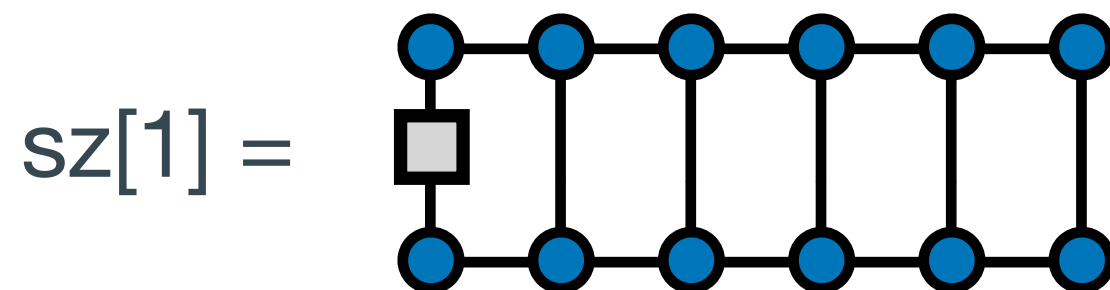
If we have an MPS and want expected values of local operators, can use **expect** function

$\Psi =$  could be output of DMRG, TDVP

Then, for example, calling expect gives

`sz = expect(psi, "Sz")`

where `sz` is an array such that



etc.

MPS Algorithms

Of course, can use various operators for spins, particles, or qubit sites

Some examples:

`sz = expect(psi, "Sz")`

magnetization of spins

`density = expect(psi, "N")`

density of particles

`Xvals = expect(psi, "X")`

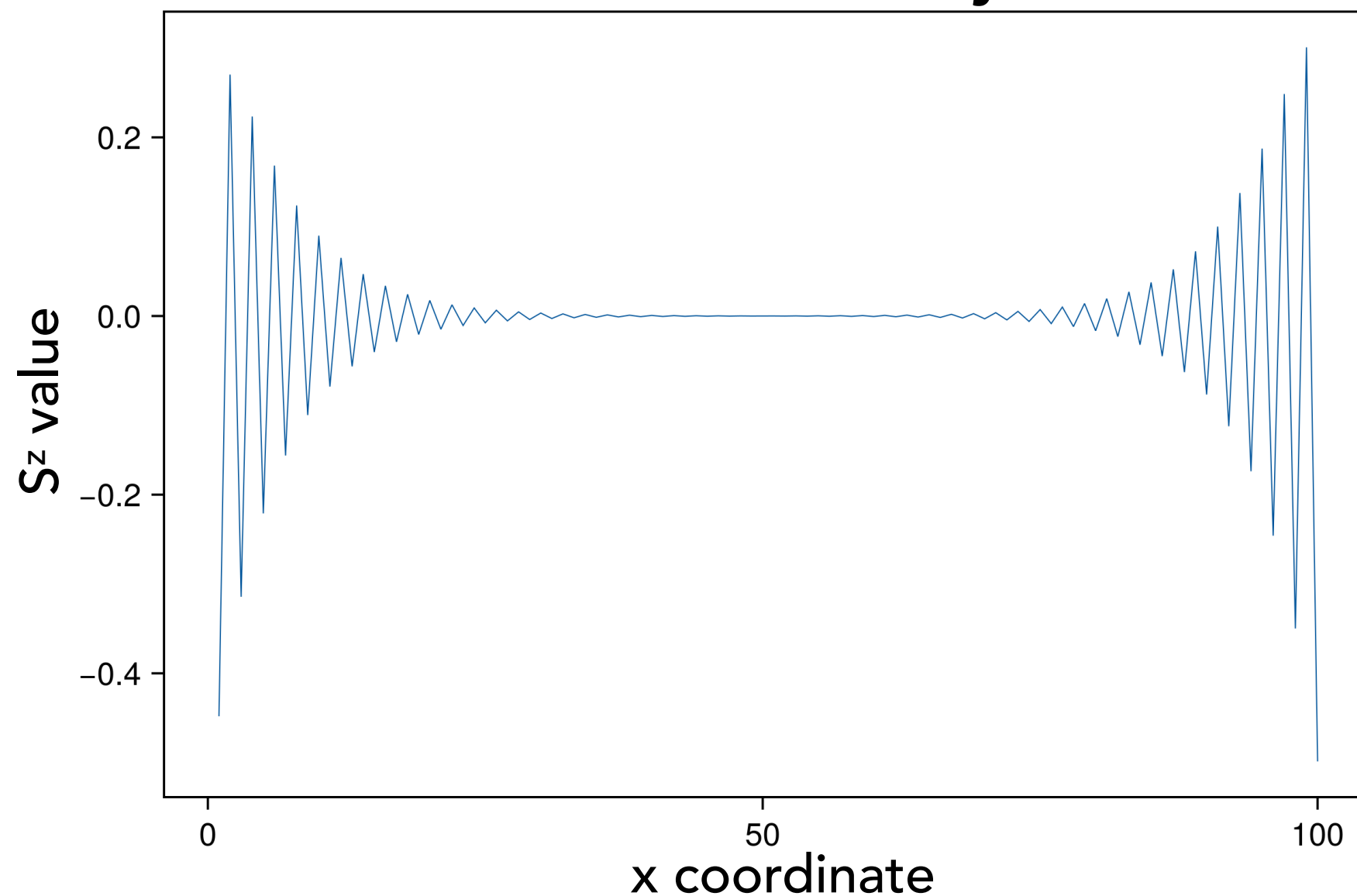
$\langle X \rangle$ over a set of qubits

MPS Algorithms

Example output:

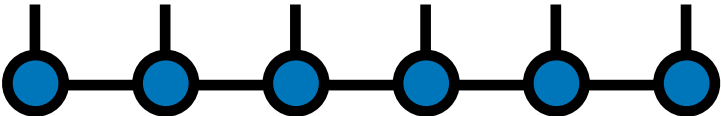
```
energy, psi = dmrg(H, psi0; nsweeps, maxdim, cutoff)  
sz = expect(psi, "Sz")
```

Plot of sz array



MPS Algorithms

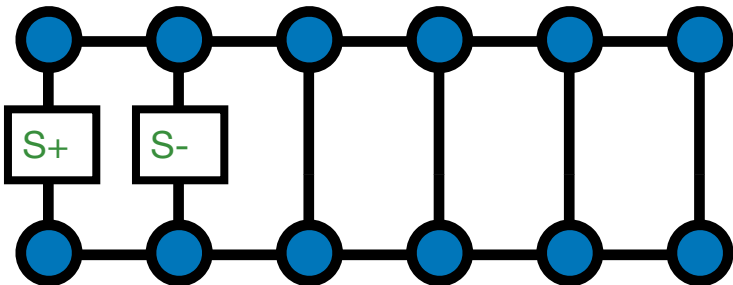
The **correlation_matrix** function also computes expected values but of "two point" functions i.e. correlators

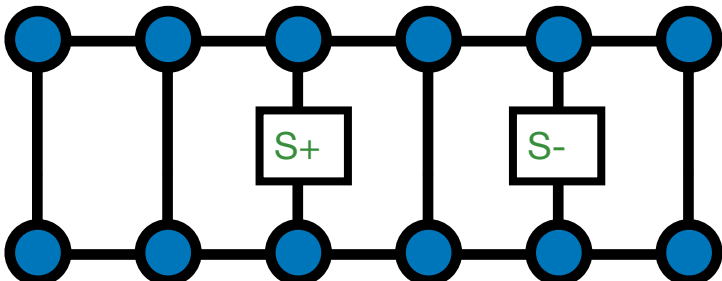
$\Psi =$  could be output of DMRG, TDVP

Then, calling `correlation_matrix` gives

`spm = correlation_matrix(psi, "S+", "S-")`

where `spm` is an Matrix such that

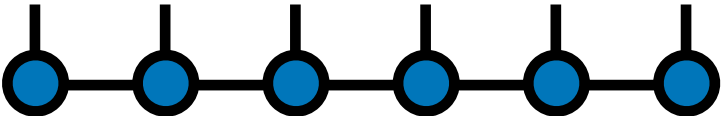
`spm[1,2] =` 

`spm[3,5] =` 

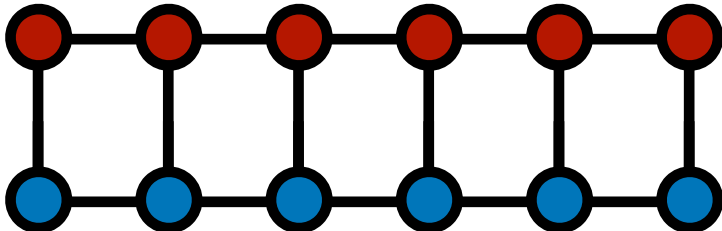
etc.

MPS Algorithms

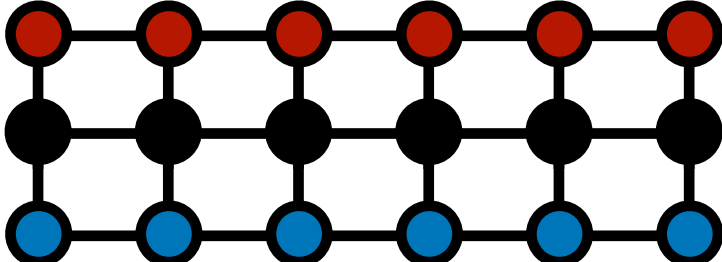
The **inner** function lets us analyze MPS through overlaps with other MPS and MPOs

$\Psi =$  could be output of DMRG, TDVP

Then, calling inner with another MPS gives

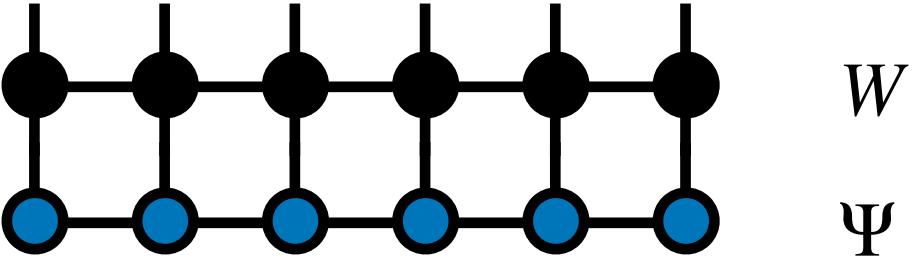
`inner(psi,phi) =`  Φ
 Ψ

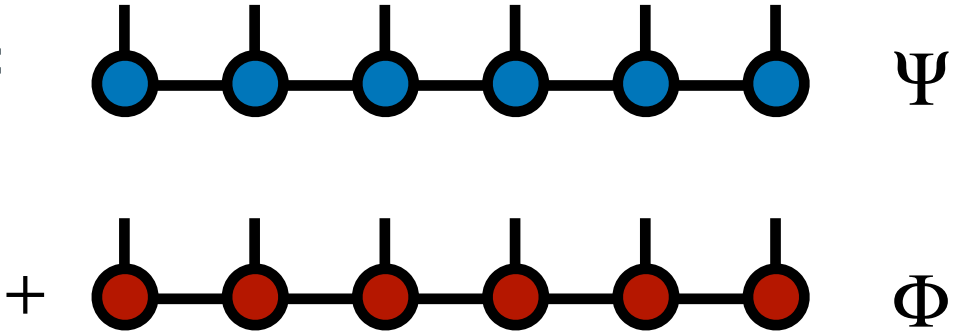
Or including an MPO like

`inner(phi',H,psi) =`  Φ
 H
 Ψ

MPS Algorithms

Finally MPS and MPO tensor networks can be contracted and added with **contract** and **sum**

$$\text{contract}(W, \text{psi}; \text{cutoff}) =$$


$$\text{sum}(\text{psi}, \text{phi}; \text{cutoff}) =$$


ITensor Advanced Features & ITensor Ecosystem

GPU Support

Easily run on GPUs by 'adapting' data type to CuArray

```
using ITensors, ITensorMPS
using CUDA
```

```
N = 100
```

```
sites = siteinds("S=1/2", N)
```

```
terms = OpSum()
```

```
for j in 1:(N - 1)
```

```
    terms += "Sz", j, "Sz", j + 1
```

```
    terms += 1/2, "S+", j, "S-", j + 1
```

```
    terms += 1/2, "S-", j, "S+", j + 1
```

```
end
```

```
H = MPO(terms, sites)
```

```
psi0 = random_mps(sites; linkdims=10)
```

```
psi0 = adapt(CuArray{element_type}, psi0)
```

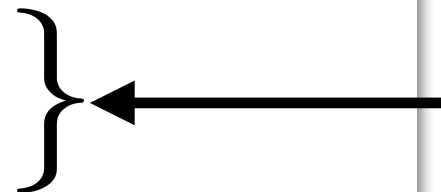
```
H = adapt(CuArray{element_type}, H)
```

```
nsweeps = 5
```

```
maxdim = [10, 20, 100, 100, 200]
```

```
cutoff = [1E-11]
```

```
energy, psi = dmrg(H, psi0; nsweeps, maxdim, cutoff)
```

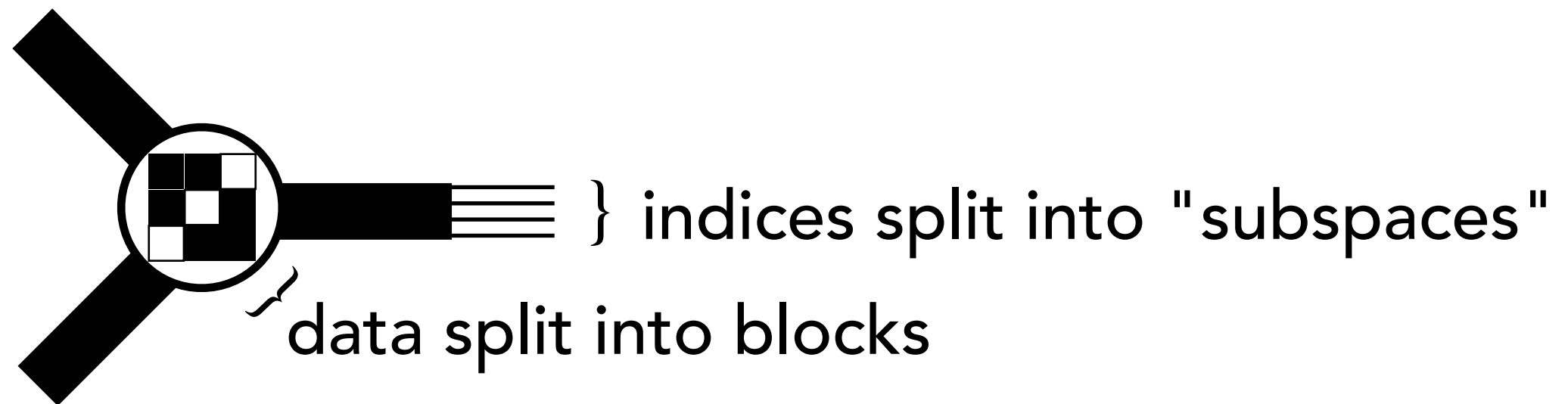


Adapting the data type

Then use in DMRG,
for example

Symmetric / Block-Sparse Tensors

Block sparse tensors in physics due to symmetries

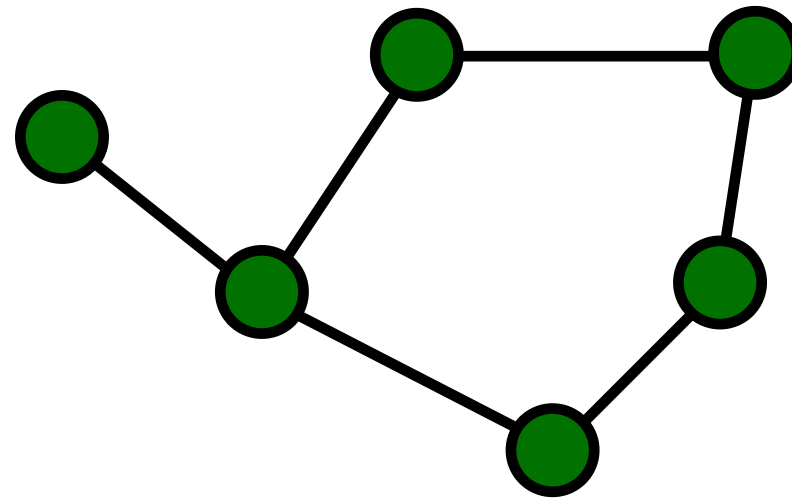
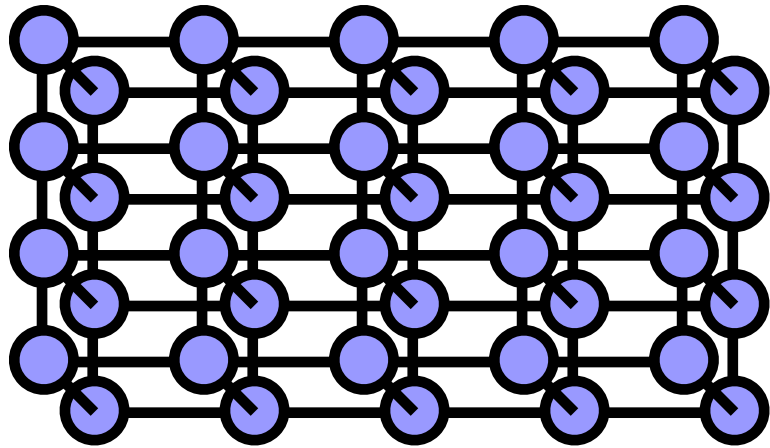


ITensor: block information *inside indices* (index objects)

```
julia> i = Index([QN("Sz",0)=>2, QN("Sz",1)=>1, QN("Sz",-1)=>1])  
(dim=4|id=444) <Out>  
1: QN("Sz",0) => 2  
2: QN("Sz",1) => 1  
3: QN("Sz",-1) => 1  
  
julia> dim(i)  
4
```

Tensor Networks

Upcoming ITensorNetworks.jl package generalizes ITensor



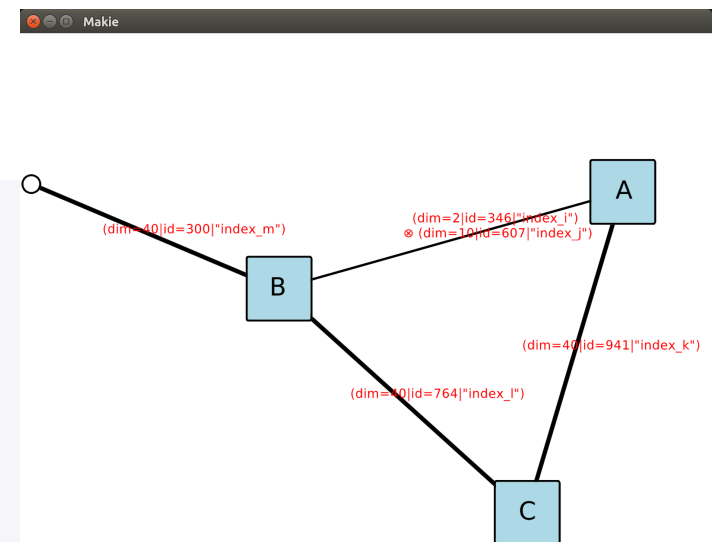
Tensor networks on
arbitrary graphs

Algorithms based on
"belief propagation"
concept

```
using ITensors
using ITensorsVisualization
```

```
i = Index(2, "index_i")
j = Index(10, "index_j")
k = Index(40, "index_k")
l = Index(40, "index_l")
m = Index(40, "index_m")
A = randomITensor(i, j, k)
B = randomITensor(i, j, l, m)
C = randomITensor(k, l)
```

```
# Contract the tensors over the common indices
# and visualize the results
ABC = @visualize A * B * C
```

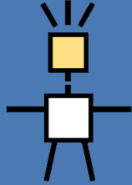


ITensor

Stay current by visiting our website & discussion forum

ITensor Documentation ▾ News Codes Discuss About ▾

ITensor

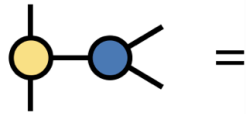


High-Performance Tensor Software
Inspired By Tensor Diagrams

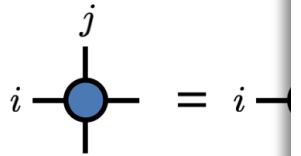
<https://itensor.org>

Code Examples

Contracting two ITensors



Factoring an ITensor



ITensor

categories ▾ tags ▾ Categories Latest Unread (220) Top

+ New Category + New Topic

Category	Topics	Latest
ITensor Julia Questions Questions about the Julia version of ITensor (ITensors.jl) and related packages .	671 137 unread	 Reduced density matrix of an MPO ITensor Julia Questions julia, mps, mpo 5 1d
DMRG and Numerical Methods Questions about applying DMRG, other MPS algorithms, and general numerical methods based on tensor networks.	245 51 unread	 Error in basic mpo contraction ITensor Julia Questions julia, mps 2 3d
General Discussion This category is for general discussion that doesn't fit into the other categories.	28 11 unread	 VUMPS convergence issues ITensor Julia Questions 6 4d
ITensor Development Discussion of the development and design of the ITensor software.	15 5 unread	 The issues of frequent recompilation and low computational efficiency in ITensor.jl ITensor Julia Questions julia 1 4d
ITensor C++ Questions	85	 Dynamical Structure Factor of 1D Spin-1 chain DMRG and Numerical Methods julia, dmrg 1 5d